

An enhanced mechanism for detection of spam emails by deep learning technique with bio-inspired algorithm

Neomi Nelin Nicholas^{a,*}, V. Nirmalrani^b

^a Faculty of Computer Science and Engineering, Sathyabama Institute of Science and Technology, Chennai, India

^b Department of Computer Science and Engineering, Sathyabama Institute of Science and Technology, Chennai, India

ARTICLE INFO

Keywords:

Tokenization
Bag of words
Curse of dimensionality
Sand Cat Swarm
Optimization algorithm
Email spam classification

ABSTRACT

This study aims to create a highly effective system for classifying email spam, with the key objective of improving performance and accuracy in classification. Rigorous pre-processing techniques, including lemmatization and tokenization, are applied to refine the dataset. The impact of optimal feature selection on deep learning algorithm stability with varying training datasets is investigated and to improve classifier performance, pre-stages such as feature extraction and selection using bag of words are utilized. The hybrid model is further enhanced by combining feature extraction and classification algorithms with optimization. The inclusion of multiple features, such as n-gram features, addresses the 'Curse of Dimensionality,' and optimal feature selection is employed to improve the spam detection process. The proposed system undergoes three main modifications: multiple feature extraction, feature selection, and an enhanced hybrid model, all aimed at improving spam detection accuracy. The e-mail classification phase involves mapping between training and testing sets, with deep learning algorithms utilized for feature extraction and classification.

The recently introduced Sand Cat Swarm Optimization algorithm is employed to refine weights during each epoch, enhancing accuracy and minimizing loss. The proposed system extends its capabilities to identify phishing attacks within the network, offering a comprehensive approach to email security.

Introduction

Generally speaking, spam denotes unwanted and unsolicited messages dispatched to a significant number of recipients. These messages can be transmitted through various communication channels like email, SMS, and social networks. Statistics indicate that a significant portion of messages on social networks constitute spam [1]. Proof point, a leading cyber security company, found a 35.5 % increase in social spam during the first half of 2013. The study revealed that for every seven new social media accounts, five were identified as spammers. The expanding influence and popularity of social networks have led to a growing user base, with a substantial amount of communication occurring within these platforms.

Deep learning models have been applied across various domains in computer science, addressing issues such as network traffic and malware detection. Emails serve as a regular means of communication and social interaction for many individuals. Security breaches that expose customer data enable spammers to create fake compromised email addresses, making it easier for them to send unauthorized (spam) emails.

This tactic is also employed for phishing attacks, tricking users into clicking on spam links within fraudulent emails. Numerous tools and techniques have been developed by companies to identify spam emails within a network. Organizations implement filtering mechanisms, establishing rules and configuring firewall settings to detect and prevent unsolicited emails [2]. Although electronic mail has streamlined communication for both organizations and individuals, spammers exploit it for fraudulent purposes by sending unsolicited emails.

In the 21st century, modern enterprises are extensively automated and face intense competition. To ensure their longevity, effective email communication among different branches of an organization is crucial. Dealing with spam in this context proves to be a formidable challenge, and the ongoing battle against it remains constant. Despite various anti-spam methods put forth by different organizations and Internet Service Providers (ISPs), a flawless solution has yet to be discovered, primarily because of the dynamic nature of electronic communication and the increasing sophistication of spammers [3]. The identification of spam emails typically involves categorizing incoming emails into spam and non-spam classes. While many modern spam detection systems rely on

* Corresponding author.

E-mail addresses: neomi2015@tistcochin.edu.in, neominicholas88@gmail.com (N.N. Nicholas).

machine learning, a common challenge is determining the optimal subsets of input features for chosen classifiers. Feature selection techniques are commonly used to tackle this issue, yet they often encounter difficulties due to the extensive data dimensions involved. This can adversely impact the performance of classifiers like SVM [4], ANN [5], and the Naïve Bayes Classifier [6]. To overcome the hurdle of high data dimensionality, reducing the feature space becomes crucial by minimizing the number of data features. However, it's vital to ensure that the feature selection process accurately picks features that represent the challenges posed by the document. The incorporation of irrelevant features not only impacts classification accuracy but also influences the duration needed for classifier training, along with feature-related expenses and the requisite number of instances for effective learning [7].

Addressing these FS-related challenges often involves leveraging evolutionary and swarm-based techniques, including ACO [8], GA [9], ABC [10], PSO [11], and HAS [12]. These methods have become widely adopted for tackling problems associated with feature selection [5,13,14]. Some employ traditional image processing methods involving manual steps such as feature extraction and selection.

Furthermore, these models rely on methodologies associated with extracting features and performing classification. In the field of computer vision, propelled by the rise of artificial intelligence, extensive studies have utilized machine learning and deep learning models to greatly enhance accuracy in recognition. The fusion of these techniques has led to remarkable strides in identifying spam, marking a significant research milestone. This advancement simplifies the automatic sorting and extraction of image traits, due to the ease brought about by machine learning and deep learning methods. Moreover, the availability of datasets, GPU resources, and software supporting advanced yet simplified deep learning structures has facilitated a transition from conventional methods to deep learning frameworks.

Recently, there has been considerable attention directed towards Convolutional Neural Networks (CNNs) due to their prowess in recognition and classification tasks. CNNs function by capturing intricate low-level characteristics from images, rendering them more favourable compared to traditional methods in automated recognition, leading to superior outcomes. A literature review reveals that existing solutions encounter challenges such as a high false-positive rate, prolonged time for identifying spam emails, and a substantial gap between the implementation of spam methods and ensuring positive outcomes. While current solutions have demonstrated noteworthy and dependable results, there remains room for enhancement. Improvements can be introduced either by proposing an entirely new methodology or by identifying weaknesses in existing solutions and suggesting remedies.

This research work falls into the second category, aiming to enhance the functionality of existing algorithms in terms of detection accuracy. The proposed algorithms seek to elevate each stage of spam detection, collectively improving the overall performance of spam detection. This article suggests an approach to identifying spam emails by employing deep learning algorithms that have been fine-tuned with bio-inspired techniques. Unlike traditional knowledge engineering, which requires manual rule-setting and constant updates for spam detection, machine learning offers an automated approach. Machine learning models learn to distinguish between undesired (spam) and genuine (ham) emails. They use this learned information to automatically sort incoming emails. This suggested approach for detecting spam aims to address the difficulty of classifying spam by exploring extra techniques, like selecting features or automatically adjusting model parameters. This research runs tests using deep learning models that integrate bio-inspired algorithms, evaluating how well they perform compared to standard models. The goal is to determine whether these suggested models enhance effectiveness through fine-tuning parameters.

Literature survey

Kumar et al. [15] assert that with the increasing number of internet

users, email spam is becoming a more significant issue, primarily due to individuals exploiting email for fraudulent, phishing, and illicit activities. Spammers create fake accounts and profiles to target unsuspecting individuals. This initiative employs machine learning methodologies to identify and filter out spam emails. The study explores various algorithms and how they can be used with different datasets. Its goal is to identify the most efficient machine learning algorithm for achieving the utmost accuracy and precision in detecting email spam. This effort aims to ensure a safer online experience by protecting users from potentially harmful connections.

Hossain et al. [16] suggested a mechanism for dividing emails into spam and ham categories by identifying extreme values using methods like DBSCAN and Isolation Forest. For comparison analysis, the model is used in deep learning and machine learning. Machine learning is implemented using ensemble techniques such as Multinomial Naïve Bayes, Random Forest, K-Nearest Neighbor, Gradient Boosting, and Recurrent Neural Networks. Artificial Neural Networks, Gradient Descent, and Recurrent Neural Networks are utilized for deep learning. Utilizing a dataset of email spam from the UCI machine learning repository, the model attains a 100 % accuracy rate in machine learning and a 99 % accuracy rate in deep learning.

Makkar et al. [17] assert that the Internet of Things, often abbreviated as IoT, constitutes a rapidly growing network wherein millions of objects are interconnected through wired or wireless data transmission channels. Although attackers frequently take advantage of flaws in machine learning algorithms, these algorithms might be extremely important in guaranteeing security and permission in IoT devices. This study introduces a structure for detecting spam in the Internet of Things using machine learning. It assesses five machine learning models with different metrics and input feature sets. The suggested scheme's efficacy in comparison to other current schemes is demonstrated by the validation findings, which are derived from the REFIT Smart Home dataset.

Sun et al. [18] outlined a Twitter spam detection system that operates in near-real-time, employing online visualizations, training models, lightweight feature extraction, and Twitter data. For the purpose of detecting spam, the system extracts features based on accounts and content. Nine popular algorithms and 1.5 million public tweets are used to train the models. To cut down on model updating expenses and training time, a parallel computing technique was presented. Additionally, the system serves as a tool for gathering tweets, which enables researchers to evaluate how well-trained classifiers perform in real-world situations.

AbdulNabi et al. [19] presented the BERT word embedding model, which separates spam emails from non-spam ones. To comprehend text context, the model makes use of attention layers. The outcomes are contrasted with two open-source data sets, a baseline DNN model, and the traditional classifiers k-NN and NB. Achieving a remarkable 98.67 % accuracy and an F1 score of 98.66 %, the model surpasses its counterparts, highlighting the efficacy of word embedding in the classification of spam emails.

Baaqel et al. [20] presented how Short Message Service (SMS) has become so popular that spammers are taking advantage of it to reach more people. There is a need for an accurate spam filtration solution because of the potential for serious consequences. Machine learning algorithms have demonstrated potential in categorizing data as either spam or ham. This paper presented a hybrid system to increase the accuracy and F-measures of spam filtration by utilizing both supervised and unsupervised machine learning techniques. This strategy seeks to enhance overall spam prevention while addressing the gravity of SMS spam filtration problems.

Alsaffar et al. [21] evaluated the effectiveness of a deep learning algorithm (RNN) and six machine learning algorithms (RF, NB, BN, SVM, KNN, and MLP) in detecting spammers within tweets. When comparing RF to the other algorithms, the findings indicate that RF consistently yields superior results, exhibiting the lowest error rate and the highest classification accuracy rate across percentage split tests and

cross-validation.

Research methodology

In this research, the dynamic landscape of communication mediums, evolving through software tools and infrastructure innovations, has spurred an increased focus on methods enhancing communication processes and fortifying defenses against malicious attacks. The escalating intricacies in spam creation have prompted companies to adopt sophisticated techniques, driving the need for improved accuracy in email spam detection. To address these challenges, a comprehensive research methodology is employed. The study commences by gathering data and making use of the Spam base dataset sourced from the UCI Machine Repository. Each attribute corresponds to a specific variable, representing individual database tables and statistical data. Rigorous pre-processing techniques, including lemmatization and tokenization, are applied to refine the dataset, ensuring optimal quality for subsequent analysis. The research introduces a hybrid model that combines feature extraction and classification algorithms with optimization to enhance spam detection. Unlike conventional spam filters that often extract a single feature category, such as colour or shape features, this work proposes the use of multiple features, specifically n-gram features, to improve the spam detection process. However, the introduction of multiple features raises the challenge of the 'Curse of Dimensionality.' This is mitigated by incorporating optimal feature selection after extraction and before classification, removing irrelevant and redundant features that could degrade performance.

Deep learning algorithms play a crucial role in this classification, involving two main steps: feature extraction and classification. The Sand Cat Swarm Optimization (SCSO) algorithm is a novel metaheuristic optimization method that mimics the hunting actions of sand cats. It optimizes weights throughout each epoch, aiming to boost accuracy and reduce loss. The optimizer, a key component, adjusts neural network attributes like learning rate and weights. In brief, the research methodology involves gathering datasets, preprocessing them, introducing a new hybrid model, applying deep learning algorithms, and using optimization techniques to improve spam detection accuracy and tackle security threats in communication networks. The block diagram for the proposed model has been shown in Fig. 1.

Data pre-processing

Preparing data is a crucial stage within the data analysis pipeline,

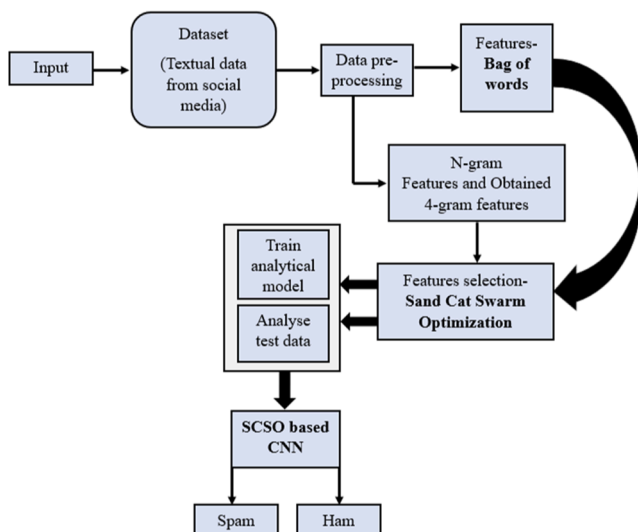


Fig. 1. Block diagram.

encompassing multiple essential tasks aimed at guaranteeing the dataset's quality and dependability. This multifaceted process encompasses data cleaning, where missing values are addressed, noisy data is smoothed out, outliers are identified or eliminated, and inconsistencies are resolved. Additionally, data integration plays a pivotal role by incorporating various databases, information files, or data sets to create a comprehensive and unified dataset. Transformation actions, such as aggregation and normalization, are applied to standardize values and make them more amenable to analysis. Moreover, data reduction strategies are employed to generate a condensed summary of the dataset, significantly reducing its size while preserving equivalent analytical outcomes. Together, these stages of data pre-processing form a robust framework for enhancing the overall quality, coherence, and usability of the data, laying a solid foundation for subsequent analyses and insights.

Tokenization

Tokenization is a crucial process in spam email detection, involving the segmentation of continuous text into meaningful units or tokens, such as words or symbols. The resulting list of tokens becomes instrumental in subsequent tasks like content mining and parsing, playing a pivotal role in both semantic and lexical analysis within the realms of computer science and engineering. In the context of spam email detection, defining what constitutes a "word" is challenging, as tokenization operates at the word level. Typically, tokens are identified based on simple heuristics, such as being separated by whitespace characters like "line breaks" or "spaces," or by the presence of "punctuation" characters. Alphabetic characters and numbers in immediate proximity form a single token, and the inclusion of white spaces and punctuation in the resulting token lists can vary. This tokenization process is integral to extracting meaningful features from email content, aiding in the identification of patterns and characteristics associated with spam, and ultimately contributing to more effective spam filtering mechanisms.

Stop word removal

After tokenizing the dataset, the next phase involves eliminating duplicate elements such as spaces, commas, periods, colons, semicolons, and additional punctuation marks. Stop words, which are fundamental English words with minimal impact on a sentence's essence, can be excluded without changing its core meaning.

In scenarios like search queries, these words, including common ones like 'how,' 'to,' and 'a,' are often disregarded to allow the search engine to prioritize and focus on more significant terms. By excluding stop words, the search engine can concentrate on retrieving pages containing specific keywords, ensuring a more targeted and relevant outcome. This practice enhances the efficiency of the search process by streamlining the emphasis on content-carrying words and improving the precision of the results obtained.

Stemming

During this procedure, the stemmed words undergo a transformation into a more convenient format. The extraction of attributes, classes, or variables is performed by utilizing the initial data. This not only contributes to the generation of more precise outputs but also significantly reduces the time required for model creation when the feature extraction process is executed effectively.

Feature extraction using BoW

Sentence generation involves the creation of sentences using natural language generation techniques, requiring a deep understanding of syntax, semantics, ontology, morphology, and other linguistic aspects. In this context, the objective is to generate sentences based on a given input of a bag of words. The bag of words consists of unordered words, the fundamental units of a sentence, derived from a grammatically correct sentence. The approach employed for sentence generation relies on a rule-based technique that utilizes linguistic rules and the

knowledge of language syntax. By exploiting the inherent interdependencies between words in a language, this method aims to construct coherent and meaningful sentences. The syntax of a language plays a crucial role in determining the arrangement of words within a sentence to effectively convey its intended meaning. It is noteworthy that different languages exhibit distinct preferred word orders. For instance, English follows the Subject-Verb-Object (SVO) word order [2]. The sentence generation process leverages these linguistic principles to produce well-structured and grammatically sound sentences.

The N-gram language model calculates the probability of the next word $p(w)$ by considering a specified context window size, denoted as N . This model determines the probability of the N th element by evaluating the preceding $(N-1)$ elements in the context. The fundamental equation for calculating the probability of the next word, based on a given context of $(N-1)$ elements, is expressed as follows (Eq. (1)).

$$p(w_n | w_1^{n-1}) \approx p(w_n | w_{n-N+1}^{n-1}) \quad (1)$$

For bigram model, the probability of next word is given by (Eq. (2)).

$$p(w_i | w_1^{i-1}) \approx p(w_i | w_{i-1}) \quad (2)$$

The N-gram model focuses on capturing local dependencies while encoding essential linguistic information, such as syntax and semantics [1]. Operating with a window size of N , the model relies on the context of $(N-1)$ words to predict the next word, making it inherently local dependent. In our approach, we employed a bigram model to construct a probability matrix from the text corpus. Using the probability matrix, we generated permuted sentences from a given bag of words through the application of a Depth-First Search (DFS) technique. Notably, each sentence in the text corpus is encapsulated within a starting symbol «s» and an ending symbol «/s» for proper representation. This technique effectively allocates some probability mass to zero probability N-grams, enhancing the model's robustness and performance.

SCSO-based CNN classifier

The Sand Cat Swarm Optimization (SCSO) algorithm draws inspiration from the natural behaviours exhibited by sand cats. In their habitat, sand cats primarily engage in foraging for prey and executing attacks. This algorithm specifically mirrors the sand cat's remarkable talent in detecting low-frequency noises, enabling them to efficiently locate prey both above and below ground. Leveraging this distinctive skill, which aids in swift prey detection and capture, is the cornerstone of this proposed algorithm. While these felines typically lead solitary lives in the wild, the creators of the algorithm envision them forming groups to highlight the concept of swarm intelligence. During the initialization phase, the algorithm determines the number of sand cats to optimize either a minimization or maximization problem. The first stages encompass forming the population and outlining the issue, assuming that sand cats, known for their solitary behaviour, function together within a swarm intelligence framework in the algorithm.

Initial population

In solving an optimization problem, it is crucial to define the values of pertinent variables in alignment with the current problem's solution. For instance, in Particle Swarm Optimization (PSO), these values are denoted as particle positions, while in Grey Wolf Optimization (GWO), they are referred to as grey wolf positions. Our proposed Sand Cat Swarm Optimization (SCSO) algorithm utilizes the concept of "sand cats," with each cat symbolizing the variables of the problem. Within our algorithm structured as a population-based method, these variables are depicted as a vector. In an optimization problem with d dimensions, a sand cat is portrayed as a $1 \times d$ array that represents the solution to the problem. Each variable value ($x_1, x_2, \dots, x_d$) is a floating-point number, with each x constrained within the specified lower and upper boundaries ($\forall x_i \in [\text{lower}, \text{upper}]$).

The SCSO algorithm begins by creating a candidate matrix that represents the Sand Cat population, shaped by the problem's scale ($N_{\text{pop}} \times N_d$), with 'pop' ranging from 1 to n . Moreover, the fitness of each Sand Cat is calculated using a specific fitness function that encompasses the key problem factors. The goal of SCSO is to iteratively optimize these parameters, aiming for their best values. As part of this process, each Sand Cat produces a value for the function it's associated with, which influences the assessment of its fitness.

At the conclusion of every iteration, the sand cat displaying the most optimal cost thus far is recognized as the leading solution, granted there is not a superior solution from previous iterations. Following this, other Sand Cats strive to align their positions with this high-performing cat in the next iteration. This method draws inspiration from the idea that similar to a cat closing in on its prey during a hunt, the initial solution in every cycle reflects proximity to the goal. If no improved solution arises in later cycles, that specific iteration's solution is not stored unnecessarily in memory, encouraging optimal memory use. This cycle repeats with each iteration of the algorithm.

Exploration

In this section, we elucidate the searching mechanism employed by the SCSO algorithm. Sand Cats rely on emitting low-frequency noise for their prey search. Each Sand Cat's solution is denoted as $X_i = (x_{i1}, x_{i2}, x_{i3}, \dots, x_{id})$. The SCSO algorithm leverages the Sand Cats' ability to detect low frequencies, with the sensitivity range for each cat being formally defined. As previously mentioned, Sand Cats can sense low frequencies below 2 kHz. In the mathematical model, the value ($\vec{r}_G$) in Eq. (3) linearly decreases from two to zero as iterations progress, aligning with the algorithm's operation to effectively approach the sought-after prey without losing or bypassing it. Consequently, the sensitivity range for prey searching assumes a starting point at 2 kHz and diminishes to 0.

The S_M value, inspired by sand cats' hearing characteristics, is set to 2. However, for different problem-solving scenarios, this value can be customized to determine the agents' action speed. This exemplifies the equation's adaptability and versatility. For example, when the maximum iteration count is set at 100, $\vec{r}_G$ surpasses 1 in the first half of the iterations but drops below 1 in the final 50 iterations. Importantly, the critical factor dictating the shift from exploration to exploitation stages is denoted by $\vec{R}$. This $\vec{R}$, derived from Eq. (4), plays a role in creating a well-rounded and flexible strategy that amplifies the transitions and opportunities in both phases.

The search space is randomly initialized within predefined boundaries. During the search step, the position update of each current search agent is based on a random position, enabling the exploration of new spaces within the search domain. To avoid falling into local optimum traps, the sensitivity range for each sand cat differs, as specified by Eq. (5). Here, $\vec{r}_G$ signifies the overall sensitivity range, which linearly decreases from 2 to 0, while $\vec{r}$ represents the sensitivity range of each cat. The latter is utilized in operations during exploration or exploitation phases, guided by $\vec{r}_G$ to control transitions in these phases. Additionally, iter_c denotes the current iteration, and iter_{Max} is the maximum number of iterations.

$$\vec{r}_G = S_M - \left(\frac{2 \times S_M \times \text{iter}_c}{\text{iter}_{\text{Max}} + \text{iter}_{\text{max}}} \right) \quad (3)$$

$$\vec{R} = 2 \times \vec{r}_G \times \text{rand}(0, 1) - \vec{r}_G \quad (4)$$

$$\vec{r} = \vec{r}_G \times \text{rand}(0, 1) \quad (5)$$

Every search agent (sand cat) adjusts its position by considering the best candidate position ($\vec{\text{Pos}}_{bc}$) vector, its current position ($\vec{\text{Pos}}_c$) vector, and its sensitivity range ($\vec{r}$). This enables sand cats to identify potential alternative prey positions (as per Eq. (6)). The equation introduces an opportunity for the algorithm to discover new local optima within the

search area. Consequently, the resulting position lies between the current position and the prey's position. Importantly, this positioning is a result of randomness rather than precise methods. The algorithm leverages this randomness to benefit the search agents, making it cost-effective and efficient in complexity.

$$\vec{Pos}(t+1) = \vec{r} \cdot (\vec{Pos}_{bc}(t) - \text{rand}(0,1) \cdot \vec{Pos}_c(t)) \quad (6)$$

Attacking on the prey (exploitation)

As previously noted, Sand Cats rely on their ear sensitivity to detect prey. To formulate the attacking phase of the Sand Cat Search Optimization (SCSO) algorithm mathematically, the distance between the best position ($\vec{Pos}_b$) (best solution) and the current position ($\vec{Pos}_c$) of the Sand Cat is computed using Eq. (7). The sensitivity range of the Sand Cat is considered a circular area, determining the direction of movement by a random angle (θ) on the circle. Other parameters, specified in Eq. (7), also influence the direction of movement. The randomly chosen angle, ranging from 0 to 360°, is mapped to the interval of -1 to 1. Consequently, each member of the population can move in a distinct circular direction within the search space.

The Sand Cat Search Optimization (SCSO) employs the Roulette Wheel selection algorithm, assigning a varied angle to each sand cat. This angle aids their approach to the hunting position and serves as a vital factor in avoiding traps of local optima. Integrating this random angle into Eq. (6), significantly enhances how the agents pursue their hunt and direct their movements. The variable Pos_{rnd} denotes the random position, ensuring the cats remain close to their prey. This gradual adjustment of positions across two successive iterations in SCSO showcases how the cats progressively approach their hunting target. Furthermore, a representative depiction of the SCSO's progression from the initial to the final iteration.

$$\begin{aligned} \vec{Pos}_{\text{rnd}} &= |\text{rand}(0,1) \cdot \vec{Pos}_b(t) - \vec{Pos}_c(t)|, \\ \vec{Pos}(t+1) &= \vec{Pos}_b(t) - \vec{r} \cdot \vec{Pos}_{\text{rnd}} \cdot \cos(\theta). \end{aligned} \quad (7)$$

Exploration and exploitation

Exploration and Exploitation are facilitated by the adaptive values of the parameters r_G and R in the SCSO algorithm. These parameters enable the algorithm to seamlessly transition between two distinct phases. Since the fluctuation range of the R parameter is dependent on r_G , a balanced distribution of r_G values result in a well-balanced R -value. Consequently, operations between the exploration and exploitation phases are appropriately managed based on the nature of the problem.

To elaborate, R is a random variable within the interval $[-2r_G, 2r_G]$, with r_G decreasing linearly from 2 to 0 over iterations. If R falls within the range of $[-1, 1]$, the Sand Cat can potentially occupy any location between its current position and the hunting spot. The SCSO algorithm directs search agents to prioritize exploitation when $R \leq 1$, encouraging exploration otherwise to uncover prey.

During the exploration phase, characterized by the search for prey, the diverse radii of each cat prevent them from falling into local optimum traps. This characteristic is also effective in the exploitation phase when attacking prey. Eq. (6) details how the position of each Sand Cat changes during both the exploration and exploitation phases. If $|R|$ is equal to or less than 1, the Sand Cats are guided toward hunting their prey. Otherwise, their objective is to explore new potential solutions within the global area.

The algorithm's equilibrium in behaviour, along with its exploration of diverse local areas within the global space, leads to a swift and precise convergence. As a result, it excels in tackling high-dimensional and multi-objective problems.

Typically, neural networks employ fully connected layers, establishing connections between each neuron in one layer and every neuron in the subsequent layer. This arrangement enables effective handling of correlations between any points within the training vectors, regardless

of their proximity. Conversely, Convolutional Neural Networks (CNNs) are specifically crafted to address local structures, making them less effective when essential information is not confined to a local context. The advantage of CNNs lies in their reduced parameter count in convolutional layers, enabling more efficient training compared to fully connected layers.

CNNs are particularly well-suited for image analysis, as images often exhibit local structures such as edges and gradients. However, CNNs have demonstrated success across various problem domains beyond images. Any problem characterized by a data representation emphasizing local structures is a suitable candidate for CNN application. This property extends beyond images to encompass fields like text analysis and speech analysis. In addition to convolutional layers, CNN architectures commonly incorporate pooling layers. Pooling layers, among other functions, facilitate down sampling from a given layer, enhancing the efficiency of subsequent computations.

Result and discussion

The implementation of the proposed model involves a carefully selected set of hardware and software specifications to ensure optimal performance. The hardware specifications include a Ryzen 5/6 series CPU, NV GTX for graphical processing, a 1 TB HDD for storage, and the Windows 10 Operating System. On the software side, the implementation leverages PyTorch, an open-source Python library tailored for constructing deep learning models. Additionally, Google Colaboratory, an open-source environment provided by Google for developing deep learning models, is utilized for its collaborative and cloud-based capabilities.

The proposed model undergoes validation using K-fold validation with an epoch rate set at 10. The learning rate is configured at 0.09, and the Stochastic Gradient Descent (SGD) optimizer is employed. The K-fold validation process helps ensure the model's robustness and generalization by partitioning the dataset into K subsets, training the model on K-1 folds, and validating the remaining fold. This process is repeated K times, providing a comprehensive evaluation of the model's performance. To assess the effectiveness of the proposed model, various performance metrics are employed, including accuracy, precision, recall, F1-score, and detection rate. These metrics collectively provide a thorough understanding of the model's classification capabilities, sensitivity, and overall predictive accuracy. Furthermore, the proposed model is benchmarked against other state-of-the-art models using a comparative evaluation under various performance measures. This comparative analysis helps gauge the efficacy of the proposed model relative to existing approaches, providing insights into its strengths and areas for potential improvement.

The implementation method integrates specific hardware and software specifications to support the proposed model's development and validation. The use of PyTorch and Google Colaboratory facilitates efficient deep-learning model construction and collaborative development. The validation process includes K-fold validation with a defined epoch rate and learning rate, coupled with the evaluation of performance metrics. Additionally, the proposed model undergoes a thorough comparative analysis against other state-of-the-art models to establish its relative effectiveness and potential contributions to the field.

Dataset description

The Spam base Dataset, available on the UCI Machine Learning Repository, is a widely utilized collection of email messages designed for spam email classification research. Comprising 58 attributes, these features encapsulate word and character frequencies, providing a rich set of characteristics for each email instance. Each instance is labelled as either spam or non-spam, enabling supervised machine-learning tasks. With a moderate size, the dataset serves as a benchmark for developing and evaluating spam filtering algorithms, making it a valuable resource

in research and educational settings. Researchers commonly split the dataset into training and testing sets to assess the performance of machine learning models in accurately discerning spam and non-spam emails based on the provided features.

Performance metrics

The Fig. 2 depicting word count versus frequency for Document 1 showcases a consistent occurrence of 5900 for word counts between 0 and 1700, suggesting a strong and continual presence of specific content within that range. However, beyond 1700 words up to 3000 words, the frequency abruptly declines to 100, indicating a notable shift or reduced repetition of content in this segment of the dataset. This abrupt decrease in frequency suggests a distinct change in the composition, subject matter, or structure of the text or dataset, leading to a considerable drop in the occurrence of those word counts. This analysis implies that the content within the initial range of up to 1700 words bears substantial repetition or similarity, while the subsequent segment showcases a marked deviation or reduced recurrence of specific word counts.

The Fig. 3 provide word count versus frequency graph for Document 2 reveals intriguing patterns within the document's content. Initially, from 0 to 1700 words, the frequency remains constant at a high value of 9960. This consistency suggests a persistent repetition or significant occurrence of specific words or content within this range, potentially highlighting a consistent theme, repetition of phrases, or structural pattern within the document's initial segment. However, the graph takes a drastic turn beyond 1700 words up to 4000 words, where the frequency plunges to a mere 10. Such a stark decline indicates a substantial shift or a notable decrease in the repetition or occurrence of words within this extended section of the document. This sudden drop in frequency could imply a significant change in topic, style, or content, diverging from the earlier recurring pattern and leading to a considerably reduced repetition of those specific word counts. Overall, the graph depicts a contrast between highly repetitive content in the document's initial portion versus a notably decreased repetition or occurrence of words in the subsequent segment.

The word count versus frequency graph for Document 3 in Fig. 4 illustrates an intriguing pattern where the frequency starts at 2600 for a word count of 0 and remains constant up to a significant word count of 0.35 million (0.35×10^6). This consistent frequency maintained across such a vast word count range indicates a sustained presence or repetition of specific words, phrases, or content within the document. This unchanging high frequency of up to 0.35 million words suggests a persistent theme, repeated sections, or consistent structural patterns throughout this extensive portion of the document. This graph

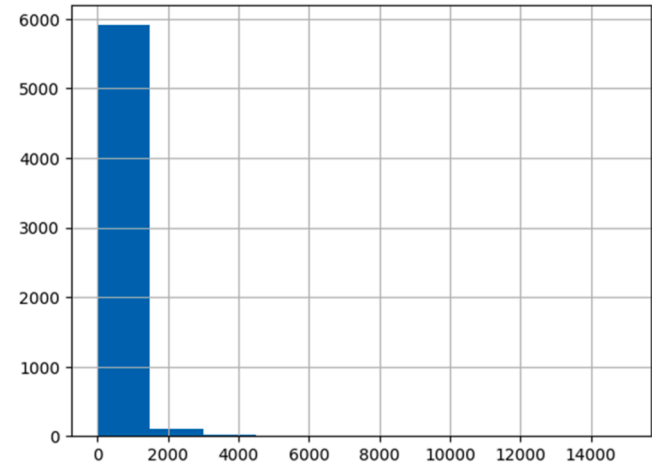


Fig. 2. Performance metrics.

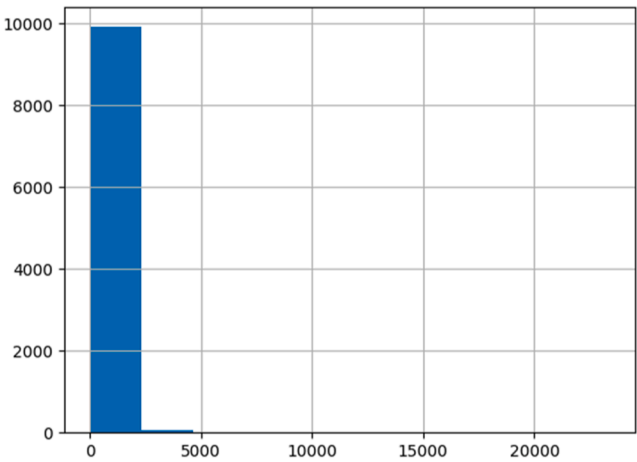


Fig. 3. Word count.

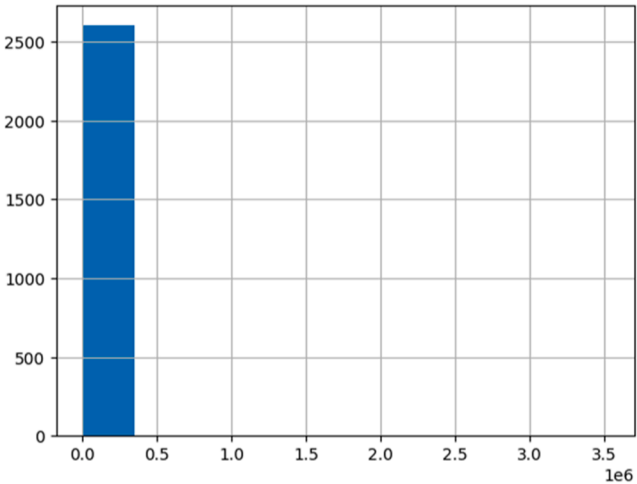


Fig. 4. Word count versus frequency graph for Document 3.

highlights a remarkable consistency and stability in the occurrence of these specific word counts, hinting at a consistent narrative or subject matter maintained over a considerable length within Document 3. The Fig. 5 shows the frequency of word count.

The combined word count versus frequency graph for the three

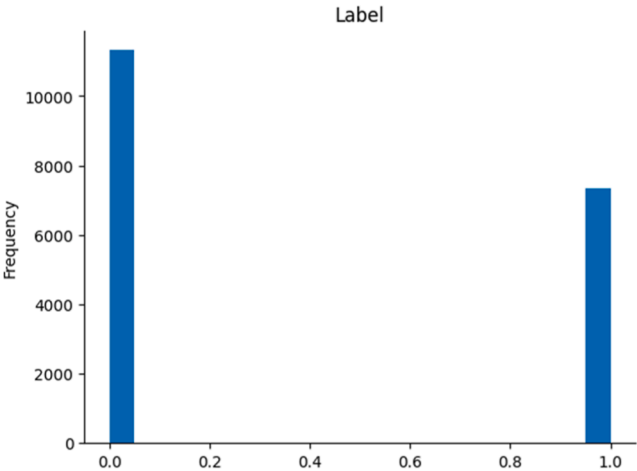


Fig. 5. Frequency in word count.

documents reveals distinctive patterns within the composite data. Initially, from 0 to 0.05-word count, the frequency remains constant at 1200. This consistency might represent commonalities or recurring elements shared among the beginnings or introductions of the three documents. However, at the higher range from 0.95 to 1 word count, there is a significant surge in frequency to 7820. This abrupt spike suggests a concentrated occurrence or repetition of specific words, phrases, or content precisely towards the end or concluding sections of these documents. This change in frequency implies a contrast between the initial portions, which share similarities, and the conclusion or final segments, where there is a distinct and notable increase in recurring content across the combined documents. Overall, the graph signifies a divergence in the occurrence of specific word counts between the beginnings and endings of these three documents when considered together.

Fig. 6 illustrates the distribution of word counts across a dataset. On the horizontal axis, word count bins are represented, and grouped into intervals, and on the vertical axis, the corresponding frequency of data points within each bin is depicted. The chart provides a visual overview of how frequently different word counts occur in the dataset. Peaks in the histogram indicate ranges of word counts that are more prevalent, while the shape of the distribution offers insights into the overall variability and concentration of word counts within the analysed data.

The graph in Fig. 7 represents a dataset consisting of 9120 labelled emails, with 1200 classified as normal mail and 7920 categorized as phishing mail. This visual likely portrays a bar chart or pie chart, delineating the distribution of these labels within the dataset. It visually demonstrates a significant disparity between the occurrences of phishing emails and normal emails, showcasing that the dataset predominantly comprises phishing emails. The graph’s visualization effectively highlights the imbalanced distribution, emphasizing that a substantial majority of the emails in the dataset are classified as phishing, while the number of normal emails is comparatively smaller.

The Table 1 provided illustrates the time taken for each epoch during the execution of a deep learning model for email spam classification within the context of a research project aimed at enhancing email security, graphically represented in Fig. 8. The research emphasizes refining the dataset through rigorous preprocessing techniques like lemmatization and tokenization. Additionally, it investigates the impact of optimal feature selection on the stability of deep learning algorithms with varying training datasets, utilizing techniques such as bag of words for feature extraction and selection. The study also involves the development and enhancement of a hybrid model by combining feature extraction and classification algorithms with optimization methods. Specifically, it explores multiple feature extraction, feature selection,

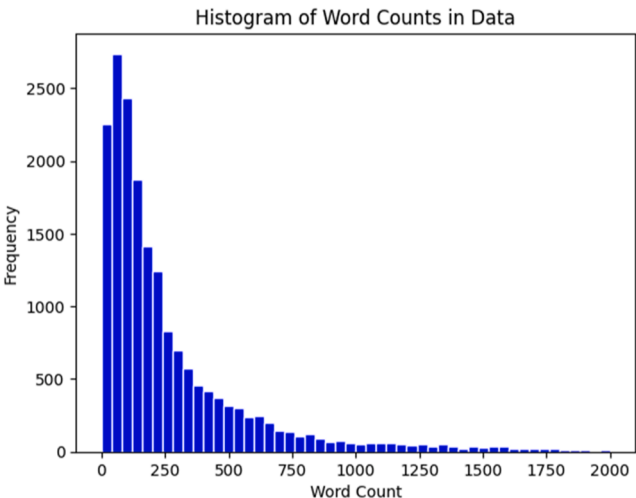


Fig. 6. Histogram view.

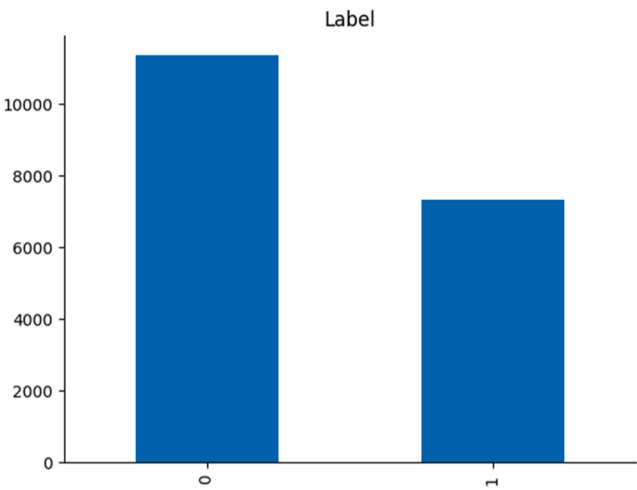


Fig. 7. Dataset labels.

Table 1 Epoch and time.	
Epoch	Time
1	4 s 659ms
2	1 s 241ms
3	1 s 240ms
4	1 s 252ms
5	6 s 2s
6	5 s 925ms
7	2 s 284ms
8	1 s 242ms
9	1 s 243ms
10	1 s 232ms

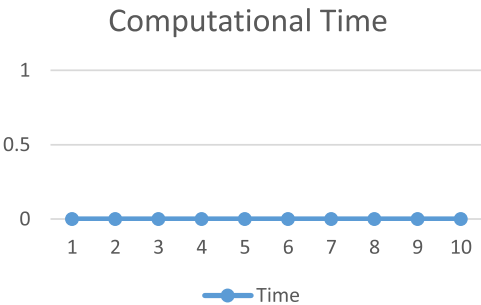


Fig. 8. Computational time.

and an enhanced hybrid model to improve spam detection accuracy. During the email classification phase, deep learning algorithms are employed for feature extraction and classification, with the use of the Sand Cat Swarm Optimization algorithm to optimize weights in each epoch, aiming to enhance accuracy and minimize loss. Moreover, the system extends its capabilities beyond spam detection to identify phishing attacks within the network, demonstrating a comprehensive approach to email security. The provided table shows the varying time durations (in seconds and milliseconds) for each epoch during the execution of the email spam classification model, which is part of this extensive research effort to bolster email security measures.

The provided Table 2 represents the training accuracy and training loss metrics across different epochs during the training phase of a machine learning model—specifically, an email spam classification system, as previously described in the research context. Each row corresponds to a specific epoch number, with columns detailing the training accuracy and training loss achieved at that particular epoch. The training

Table 2
Training accuracy and loss.

Epoch	Training Accuracy	Training Loss
1	0.7312	0.6442
2	0.9812	0.2257
3	0.9875	0.0950
4	0.9812	0.0517
5	0.9875	0.0346
6	0.9875	0.0268
7	0.9875	0.0233
8	0.9875	0.0212
9	0.9875	0.0199
10	0.9875	0.0191

accuracy indicates the proportion of correctly classified instances during the training phase, reaching high levels around 98.75 % consistently from epoch 3 to epoch 10, indicating a stable and high accuracy rate achieved during the model training. Simultaneously, the training loss, which measures the model’s error during training, progressively decreases from epoch 1 to epoch 10, starting at 0.6442 and continually reducing to 0.0191 by epoch 10. This decline signifies the model’s improved performance in minimizing errors and optimizing its predictive capability throughout the training process, indicating a successful convergence towards lower training loss values over successive epochs. Overall, the table illustrates the model’s progressively improving accuracy and decreasing loss, reflecting its enhanced learning and predictive performance over multiple training epochs. The training performance is represented in Fig. 9.

The Table 3 presents the validation loss and validation accuracy metrics across different epochs in the context of an email spam classification system, aligning with the research abstract previously described. Each row corresponds to a specific epoch during the validation phase, detailing the validation loss and validation accuracy achieved at that epoch. The validation loss represents the error of the model’s predictions on a separate validation dataset during training, progressively decreasing from 0.5657 in epoch 1 to 0.2637 in epoch 10. This declining trend indicates the model’s improvement in making more accurate predictions as the training progresses, signifying the optimization of the model’s performance in minimizing errors in the validation data. Simultaneously, the validation accuracy, depicting the proportion of correctly classified instances on the validation set, remains consistently high at 92.50 % across all epochs (from epoch 2 to epoch 10), suggesting a stable and high level of accuracy achieved during the model’s validation phase. This table illustrates that as the training advances through multiple epochs, the model improves its predictive accuracy and decreases errors on the validation dataset, aligning with the goal of enhancing the email spam classification system’s performance. The validation performance is represented in Fig. 10.

Conclusion

In conclusion, this research delves into the domain of email spam detection, leveraging the widely used Spam base Dataset available on the UCI Machine Learning Repository. The dataset, consisting of labelled instances with 58 attributes representing word and character frequencies, serves as a foundational resource for developing and testing spam filtering algorithms. The research methodology involves pre-processing textual data, creating a histogram to visualize word count frequencies, and implementing machine learning models for classification. The models undergo validation using K-fold validation, with metrics such as accuracy, precision, recall, F1-score, and detection rate employed for evaluation. Through comprehensive analyses and comparisons with state-of-the-art models, this research aims to contribute insights and advancements to the field of email spam filtering. Ultimately, the study emphasizes the significance of robust datasets, effective feature extraction techniques, and the integration of diverse

Training Performance

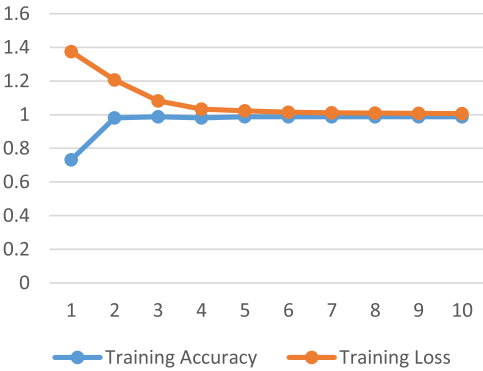


Fig. 9. Training performance.

Table 3
Validation accuracy and loss.

Epoch	Validation Loss	Validation Accuracy
1	0.5657	0.8250
2	0.4606	0.9250
3	0.3908	0.9250
4	0.3427	0.9250
5	0.3114	0.9250
6	0.2906	0.9250
7	0.2779	0.9250
8	0.2698	0.9250
9	0.2667	0.9250
10	0.2637	0.9250

Validation performance

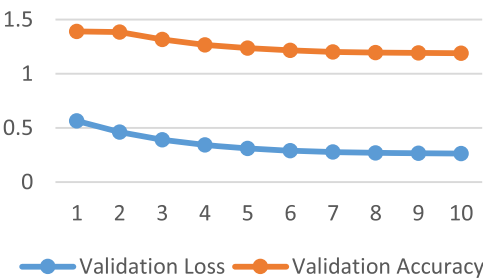


Fig. 10. Validation performance.

methodologies for enhancing the accuracy and efficiency of spam detection systems.

CRedit authorship contribution statement

Neomi Nelin Nicholas: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **V. Nirmalrani:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] T.O. Omotehinwa, D.O. Oyewola, Hyperparameter optimization of ensemble models for spam email detection, *Appl. Sci.* 13 (3) (2023) 1971.
- [2] Y. Guo, Z. Mustafaoglu, D. Koundal, Spam detection using bidirectional transformers and machine learning classifier algorithms, *J. Comput. Cogn. Eng.* 2 (1) (2023) 5–9.
- [3] J. Doshi, K. Parmar, R. Sanghavi, N. Shekokar, A comprehensive dual-layer architecture for phishing and spam email detection, *Comput. Secur.* 133 (2023) 103378.
- [4] R. Sharma, G. Kaur, E-mail spam detection using SVM and RBF, *Int. J. Mod. Educ. Comput. Sci.* 8 (4) (2016) 57.
- [5] R. Talaie Pashiri, Y. Rostami, M. Mahrami, Spam detection through feature selection using artificial neural network and sine-cosine algorithm, *Math. Sci.* 14 (2020) 193–199.
- [6] K. Agarwal, T. Kumar, Email spam detection using an integrated approach of Naïve Bayes and particle swarm optimization, in: *Proceedings of the Second International Conference on Intelligent Computing and Control Systems (ICICCS)*, IEEE, 2018, pp. 685–690.
- [7] M. Sasaki, H. Shinnou, Spam detection using text clustering, in: *Proceedings of the International Conference on Cyberworlds (CW'05)*, IEEE, 2005, p. 4. -pp.
- [8] D.K. Renuka, P. Visalakshi, T. Sankar, Improving E-mail spam classification using ant colony optimization algorithm, *Int. J. Comput. Appl.* 22 (2015) 22–26.
- [9] I.A. Taloba, S.S.I. Ismail, An intelligent hybrid technique of decision tree and genetic algorithm for e-mail spam detection, in: *Proceedings of the Ninth International Conference on Intelligent Computing and Information Systems (ICICIS)*, IEEE, 2019, pp. 99–104.
- [10] A. Singh, N. Chahal, S. Singh, S.K. Gupta, Spam detection using ANN and ABC algorithm, in: *Proceedings of the 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, IEEE, 2021, pp. 164–168.
- [11] I. Idris, A. Selamat, Improved email spam detection model with negative selection algorithm and particle swarm optimization, *Appl. Soft Comput.* 22 (2014) 11–27.
- [12] M. Sakhidek Hovshin, A new model for email spam detection using hybrid of magnetic optimization algorithm with harmony search algorithm, *Biannu. J. Monadi Cybersp. Secur. (AFTA)* 9 (1) (2020) 39–50.
- [13] A. Joshi, P. Pattanshetti, R. Tanuja, Phishing attack detection using feature selection techniques, in: *Proceedings of International Conference on Communication and Information Processing (ICCIP)*, 2019.
- [14] H. Mohammadzadeh, F.S. Gharehchopogh, Feature selection with binary symbiotic organisms search algorithm for email spam detection, *Int. J. Inf. Technol. Decis. Mak.* 20 (01) (2021) 469–515.
- [15] N. Kumar, S. Sonowal, Email spam detection using machine learning algorithms, in: *Proceedings of the Second International Conference on Inventive Research in Computing Applications (ICIRCA)*, IEEE, 2020, pp. 108–113.
- [16] F. Hossain, M.N. Uddin, R.K. Halder, Analysis of optimized machine learning and deep learning techniques for spam detection, in: *Proceedings of the IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS)*, IEEE, 2021, pp. 1–7.
- [17] A. Makkar, S. Garg, N. Kumar, M.S. Hossain, A. Ghoneim, M. Alrashoud, An efficient spam detection technique for IoT devices using machine learning, *IEEE Trans. Ind. Inform.* 17 (2) (2020) 903–912.
- [18] N. Sun, G. Lin, J. Qiu, P. Rimba, Near real-time Twitter spam detection with machine learning techniques, *Int. J. Comput. Appl.* 44 (4) (2022) 338–348.
- [19] Q. Yaseen, Spam email detection using deep learning techniques, *Procedia Comput. Sci.* 184 (2021) 853–858.
- [20] H. Baaqeel, R. Zagrouba, Hybrid SMS spam filtering system using machine learning techniques, in: *Proceedings of the 21st International Arab Conference on Information Technology (ACIT)*, IEEE, 2020, pp. 1–8.
- [21] D. Alsaffar, A. Alfahhad, B. Alqhtani, L. Alamri, S. Alansari, N. Alqahtani, D. A. Alboaneen, Machine and deep learning algorithms for Twitter spam detection, in: *Proceedings of the International Conference on Advanced Intelligent Systems and Informatics*, Springer International Publishing, Cham, 2019, pp. 483–491.



Neomi Nelin Nicholas received her Bachelor's in Information Technology and Master's in Computer Science and Engineering from Cochin University of Science and Technology, India in 2010 and 2012 respectively. She is pursuing her Ph.D. in Computer Science and Engineering from Sathyabama Institute of Science and Technology, Chennai, India. She is currently working as Assistant Professor in the Department of Information Technology, APJ Abdul Kalam Technological University, India, with an experience of 10 years. Her current research interests include Data Analytics and Data Optimization.



Dr. V. Nirmalrani received M.C.A (Computer Application) from Bharathidasan University Trichy, India in 2000 and M. Tech (Information Technology) from Sathyabama University Chennai, India in 2007. She has received Ph.D. degree from Sathyabama Institute of Science and Technology, Chennai, India in 2018. She worked in Annai Mathammal Sheela Engineering College, Namakkal as an Assistant Professor for 4 Years and 6 months. She is currently working as an Associate Professor at Sathyabama Institute of Science and Technology. She has totally 17 Years of Teaching experience in various Engineering Colleges. She has reviewed many papers in various International Journals. She has published 30+ papers in various International and National Journals. She has also authored a Book titled "C and Data Structures". Her area of interest includes Data Science, Machine Learning, Big Data, Data Analytics, Data Mining, Network Security, etc. She is the receiver of National Bright Academic Excellence Award from Green ThinkerZ, Government of Punjab, India in 2021.